

DOCUMENTO DE ALCANCE FUNCIONAL

Sistema de Reserva de Canchas Deportivas

Pádel · Tennis · Básquet

Maestría en Ingeniería de Software

Curso: Desarrollo de Aplicaciones Empresariales

Proyecto Integrador — Definición de Alcance

2026

Tabla de Contenido

1. Introducción.....	3
2. Objetivo del Proyecto	3
2.1 Objetivos específicos	3
3. Alcance Funcional	4
3.1 Roles y permisos	4
3.2 Módulos y pantallas	4
3.3 Detalle de funcionalidades clave	5
3.4 Reglas de negocio	5
3.5 Fuera de alcance	6
4. Alcance Técnico	6
4.1 Frontend — Microfrontends con Module Federation.....	6
4.2 Backend — Microservicios con Java Spring Boot	7
4.3 Base de datos — PostgreSQL	7
4.4 Consideraciones técnicas generales	8
5. Entregables	8
6. Rúbrica de Evaluación.....	9
6.1 Niveles de logro	9
7. Criterios de Aceptación del Alcance	10

1. Introducción

Este documento define el alcance funcional y técnico del proyecto integrador del curso Desarrollo de Aplicaciones Empresariales, correspondiente a la Maestría en Ingeniería de Software. El proyecto consiste en el diseño e implementación de un sistema de reserva de canchas deportivas que permita a los estudiantes aplicar, de manera práctica, los conocimientos adquiridos en microfrontends con Module Federation y microservicios con Java Spring Boot, sobre una base de datos relacional PostgreSQL.

El alcance ha sido delimitado de forma deliberada para que sea claro, medible y alcanzable dentro del tiempo de un curso académico, priorizando la calidad de la arquitectura y la correcta aplicación de los patrones estudiados por sobre la cantidad de funcionalidades implementadas.

2. Objetivo del Proyecto

Desarrollar un sistema web de reserva de canchas deportivas (pádel, tenis y básquet) que permita a un usuario final consultar disponibilidad, crear y cancelar reservas, y a un administrador gestionar el catálogo de canchas y visualizar reportes básicos de uso; implementado bajo una arquitectura de microfrontends (Module Federation) en el frontend y microservicios (Spring Boot) en el backend, persistiendo la información en PostgreSQL.

2.1 Objetivos específicos

- Aplicar el patrón de microfrontends mediante Module Federation, integrando módulos independientes en una aplicación contenedora (shell).
- Diseñar e implementar microservicios independientes con Spring Boot, cada uno responsable de un dominio de negocio acotado.
- Modelar y persistir la información del sistema en una base de datos PostgreSQL, respetando la independencia de datos entre microservicios.
- Implementar reglas de negocio simples y verificables asociadas al proceso de reserva de canchas deportivas.
- Construir un módulo de reportes básicos que permita evidenciar el uso de la información almacenada.

3. Alcance Funcional

El sistema contempla dos roles de usuario: Usuario Final y Administrador. A continuación se detallan los módulos, pantallas y funcionalidades incluidas dentro del alcance del proyecto.

3.1 Roles y permisos

Funcionalidad	Usuario Final	Administrador	Módulo
Registro e inicio de sesión	Sí	Sí	Seguridad
Consultar disponibilidad de canchas	Sí	Sí	Reservas
Crear una reserva	Sí	No	Reservas
Cancelar una reserva propia	Sí	No	Reservas
Cancelar cualquier reserva	No	Sí	Reservas
Consultar historial de reservas propias	Sí	No	Reservas
Gestionar catálogo de canchas (crear/editar/inactivar)	No	Sí	Administración
Gestionar horarios y bloqueos de mantenimiento	No	Sí	Administración
Gestionar usuarios (activar/inactivar)	No	Sí	Administración
Visualizar reportes básicos de ocupación	No	Sí	Reportes

3.2 Módulos y pantallas

El sistema se organiza en cuatro módulos funcionales, cada uno asociado a una o más pantallas:

Módulo	Pantalla	Descripción
Seguridad	Inicio de sesión / Registro	Autenticación de usuario final y administrador.
Reservas	Consulta de disponibilidad	Calendario/grilla de horarios disponibles por cancha y deporte.
Reservas	Nueva reserva	Formulario para reservar cancha, fecha y bloque horario.
Reservas	Mis reservas	Listado de reservas del usuario con opción de cancelar.
Administración	Gestión de canchas	ABM de canchas: nombre, deporte, horario de atención, estado.
Administración	Gestión de reservas	Listado global de reservas con opción de cancelar cualquiera.
Reportes	Reportes básicos	Ocupación por cancha/deporte, reservas por período, cancelaciones.

3.3 Detalle de funcionalidades clave

3.3.1 Consulta de disponibilidad

Permite a cualquier usuario autenticado visualizar, para una cancha y fecha seleccionadas, los bloques horarios disponibles y ocupados dentro del horario de atención configurado.

3.3.2 Reserva de cancha

Permite al usuario final seleccionar una cancha (pádel, tenis o básquet), una fecha y un bloque horario disponible, y confirmar la reserva. El sistema valida que el bloque no esté ocupado antes de confirmar.

3.3.3 Cancelación de reserva

Permite al usuario final cancelar una reserva propia que aún no haya ocurrido. El administrador puede cancelar cualquier reserva del sistema, por ejemplo ante un mantenimiento imprevisto de una cancha.

3.3.4 Gestión de canchas (administrador)

Permite crear, editar e inactivar canchas, definiendo su nombre, tipo de deporte y horario de atención.

3.3.5 Reportes básicos

Módulo de solo lectura, disponible únicamente para el administrador, que presenta indicadores simples y accionables:

- Número de reservas por cancha y por deporte, en un rango de fechas.
- Porcentaje de ocupación por cancha (horas reservadas sobre horas disponibles).
- Número de cancelaciones por período.
- Listado de las canchas con mayor y menor demanda.

3.4 Reglas de negocio

Las siguientes reglas de negocio acotan el comportamiento del proceso de reserva sin añadir complejidad innecesaria al desarrollo del proyecto:

RN-#	Descripción de la regla de negocio
RN-01	Una reserva se realiza sobre una cancha específica (pádel, tenis o básquet), en una fecha y en un bloque horario predefinido (por ejemplo, franjas de 1 hora).
RN-02	No se permite crear una reserva sobre un bloque horario que ya se encuentre ocupado para la misma cancha (validación de solapamiento).
RN-03	Un usuario final solo puede cancelar sus propias reservas; el administrador puede cancelar cualquier reserva del sistema.
RN-04	Una reserva solo puede cancelarse si su fecha y hora de inicio aún no han ocurrido (no se cancelan reservas pasadas).
RN-05	Toda reserva cancelada libera automáticamente el bloque horario, permitiendo que otro usuario pueda reservarlo.

RN-#	Descripción de la regla de negocio
RN-06	Un usuario final puede tener un límite configurable de reservas activas simultáneas (por ejemplo, máximo 3), para evitar acaparamiento de horarios.
RN-07	Solo el administrador puede crear, editar o inactivar canchas y definir su horario de atención (por ejemplo, 07:00 a 22:00).
RN-08	Toda reserva queda asociada a un estado: Confirmada, Cancelada o Finalizada, para efectos de trazabilidad y reportes.

3.5 Fuera de alcance

Con el fin de mantener el proyecto acotado y realizable en el tiempo del curso, quedan explícitamente fuera del alcance funcional:

- Pasarela de pagos o cobro en línea de las reservas.
- Notificaciones automáticas por correo electrónico, SMS o push.
- Reservas recurrentes o recurrencia de horarios (por ejemplo, "todos los lunes").
- Gestión de torneos, ligas o inscripciones grupales.
- Aplicación móvil nativa (el sistema es web, responsive).
- Reportes avanzados con inteligencia de negocio (BI) o exportación a formatos analíticos.

4. Alcance Técnico

Desde el punto de vista técnico, el proyecto está orientado a que el estudiante aplique de forma práctica los conceptos de arquitectura de microfrontends y microservicios revisados en el curso. La siguiente arquitectura de referencia es de carácter orientativo; el equipo puede ajustar el nombre de los componentes manteniendo el mismo enfoque arquitectónico.

4.1 Frontend — Microfrontends con Module Federation

El frontend se construye bajo el patrón de microfrontends, utilizando Module Federation (Webpack 5 / Rsbuild) para la integración de módulos independientes en tiempo de ejecución. Se sugiere la siguiente composición:

Microfrontend	Responsabilidad	Tipo (Module Federation)
shell / host	Contenedor principal: layout, navegación, autenticación y orquestación de los remotes vía Module Federation.	Host
mf-reservas	Consulta de disponibilidad, creación y cancelación de reservas (usuario final).	Remote

Microfrontend	Responsabilidad	Tipo (Module Federation)
mf-administracion	Gestión de canchas, horarios y reservas (administrador).	Remote
mf-reportes	Visualización de reportes básicos de ocupación y uso.	Remote

Cada microfrontend debe poder desarrollarse y desplegarse de forma independiente, exponiendo únicamente los componentes necesarios para su integración en el shell.

4.2 Backend — Microservicios con Java Spring Boot

El backend se implementa como un conjunto de microservicios independientes desarrollados con Spring Boot, cada uno con su propia responsabilidad y, preferentemente, su propio esquema o base de datos dentro de PostgreSQL. Se sugiere la siguiente composición:

Microservicio (Spring Boot)	Responsabilidad	Base de datos PostgreSQL
ms-usuarios	Registro, autenticación y gestión de usuarios y roles (administrador / usuario final).	usuarios_db
ms-canchas	Catálogo de canchas, deportes y horarios de atención.	canchas_db
ms-reservas	Creación, consulta y cancelación de reservas; validación de disponibilidad y reglas de negocio.	reservas_db
ms-reportes	Generación de reportes básicos (consume datos de reservas y canchas).	Lectura desde reservas_db / canchas_db

- Cada microservicio expone una API REST documentada (OpenAPI/Swagger).
- La comunicación entre microservicios, de ser necesaria, se realiza vía HTTP/REST de forma síncrona.
- Se recomienda un API Gateway o Backend for Frontend (BFF) simple como punto de entrada único para los microfrontends, sin que esto sea un requisito obligatorio.

4.3 Base de datos — PostgreSQL

La persistencia de la información se implementa sobre PostgreSQL. Cada microservicio gestiona su propio modelo de datos, evitando el acceso directo a tablas de otro microservicio. Como referencia mínima:

- usuarios_db: usuarios, roles y credenciales.
- canchas_db: canchas, deportes y horarios de atención.

- reservas_db: reservas, con su estado (Confirmada, Cancelada, Finalizada), cancha, usuario, fecha y bloque horario.

El módulo de reportes puede consultar la información necesaria mediante llamadas a los microservicios de canchas y reservas, sin requerir un almacén de datos analítico independiente.

4.4 Consideraciones técnicas generales

- El proyecto debe ejecutarse en un entorno local (o contenedores Docker) para efectos de evaluación.
- No se exige un mecanismo de autenticación avanzado (OAuth2/SSO); es suficiente una autenticación básica con roles (usuario final / administrador).
- No se exige orquestación con Kubernetes; Docker Compose es suficiente para levantar el entorno completo.
- El diseño debe priorizar la claridad arquitectónica sobre la sofisticación técnica adicional.

5. Entregables

El proyecto se evalúa a partir de los siguientes entregables:

Código	Descripción	Formato
E1	Documento de arquitectura del proyecto (diagrama de microfrontends, microservicios y modelo de datos).	Documento (PDF/Word)
E2	Shell (host) y microfrontends funcionando de forma integrada mediante Module Federation.	Código fuente + demo
E3	Microservicios Spring Boot (usuarios, canchas, reservas, reportes) documentados con Swagger/OpenAPI.	Código fuente + colección de pruebas
E4	Scripts de creación de base de datos PostgreSQL (DDL) y datos de prueba (seed).	Scripts SQL
E5	Manual breve de despliegue e instrucciones de ejecución local.	Documento (PDF/Word)
E6	Presentación final del proyecto con demostración en vivo de los flujos principales.	Presentación + demo

6. Rúbrica de Evaluación

La evaluación del proyecto integrador se realiza sobre 100 puntos, distribuidos en los siguientes criterios:

Criterio	Descripción	Peso
Alcance funcional implementado	Disponibilidad, creación y cancelación de reservas operativas para los dos roles definidos (usuario final y administrador).	25%
Arquitectura de microfrontends	Correcta implementación de Module Federation: shell (host) y al menos dos remotes integrados y desplegables de forma independiente.	15%
Arquitectura de microservicios	Microservicios Spring Boot independientes, con responsabilidades bien delimitadas y API REST documentada (Swagger/OpenAPI).	15%
Modelo de datos y persistencia	Modelo de datos correctamente normalizado en PostgreSQL, con independencia de datos entre microservicios.	15%
Reglas de negocio y validaciones	Aplicación correcta de las reglas de negocio (RN-01 a RN-08), en especial la validación de solapamiento de horarios.	10%
Módulo de reportes básicos	Reportes de ocupación, reservas por período y cancelaciones, con datos consistentes respecto a las reservas registradas.	10%
Calidad técnica y documentación	Buenas prácticas de código, manual de despliegue y presentación clara de la arquitectura y del funcionamiento del sistema.	10%
Total		100%

6.1 Niveles de logro

Cada criterio de la tabla anterior se califica según el siguiente esquema general de niveles de logro:

Nivel de logro	Puntaje	Descripción general
Excelente	90 - 100	Cumple la totalidad de los criterios con alta calidad técnica: la arquitectura de microfrontends y microservicios está correctamente aplicada, las reglas de negocio funcionan sin errores y la documentación está completa.
Bueno	75 - 89	Cumple la mayoría de los criterios. Presenta observaciones menores en la aplicación de los patrones arquitectónicos, en alguna regla de negocio o en la documentación entregada.
Suficiente	60 - 74	Cumple el alcance funcional mínimo (reserva, consulta y cancelación), pero presenta debilidades relevantes en la arquitectura técnica, en el modelo de datos o en las reglas de negocio.
Insuficiente	Menor a 60	No cumple el alcance funcional mínimo, o el proyecto no aplica microfrontends con Module Federation y/o microservicios con Spring Boot según lo solicitado.

Nota: un proyecto que no implemente microfrontends con Module Federation, o que no implemente microservicios con Spring Boot sobre PostgreSQL, no puede calificarse por encima del nivel Suficiente en los criterios de arquitectura, independientemente de la calidad de las demás funcionalidades.

7. Criterios de Aceptación del Alcance

Se considera que el alcance funcional ha sido cumplido cuando:

1. El usuario final puede consultar disponibilidad, crear y cancelar una reserva de cualquiera de las tres canchas (pádel, tenis, básquet).
2. El administrador puede gestionar el catálogo de canchas y cancelar cualquier reserva del sistema.
3. El sistema impide crear una reserva sobre un bloque horario ya ocupado.
4. El módulo de reportes básicos muestra al menos ocupación por cancha y número de reservas por período.
5. El frontend está compuesto por al menos un shell y dos microfrontends remotos integrados mediante Module Federation.
6. El backend está compuesto por al menos dos microservicios Spring Boot independientes, persistiendo en PostgreSQL.